

SETHU INSTITUTE OF TECHNOLOGY

(An Autonomous Institution | Accredited with 'A++' Grade by NAAC)

PULLOOR, KARIAPATTI - 626 115



**DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE & MACHINE
LEARNING)**

PRACTICAL RECORD NOTEBOOK

SEMESTER: VII

R21UAD707 - NATURAL LANGUAGE PROCESSING LABORATORY

NAME:

REG. No:

DEPT & YEAR:

ACADEMIC YEAR:



SETHU INSTITUTE OF TECHNOLOGY

PULLOOR - 625 115. KARIAPATTI TALUK, VIRUDHUNAGAR DISTRICT

PRACTICAL RECORD

BONAFIDE CERTIFICATE

Certified that this the bonafide record of work done by

.....

in the

Laboratory during the year.....

Staff-in-charge

Head of the Department

Reg. No. :

Submitted for the Practical Examination held on

Internal Examiner

External Examiner

INDEX

Ex.No.	Date	Experiment	Marks	Signature
1.		WORD ANALYSIS		
2.		WORD GENERATION		
3.		MORPHOLOGY		
4.		N-GRAMS		
5.		NER WSD PARSING TECHNIQUES		
6.		POS TAGGING: HIDDEN MARKOV MODEL		
7.		POS TAGGING: VITERBI DECODING		
8.		COREFERENCE RESOLUTION WITH WORD EMBEDDINGS		
9.		CHUNKING		
10.		BUILDING CHUNKER		

INSTITUTE VISION

To achieve quality education in Computer Science and Design Engineering and research for the betterment of the society

INSTITUTE MISSION

- To provide quality technical education to fulfill the aspiration of the student and to meet the needs of the Industry.
- To provide holistic learning ambience.
- To impart skills leading to employability and entrepreneurship.
- To establish effective linkage with industries.
- To promote Research and Development activities.
- To offer services for the development of society through education and technology.

CORE VALUES

- Quality
- Commitment
- Innovation
- Team work
- Courtesy

QUALITY POLICY

- To provide Quality technical education to the students
- To produce competent professionals and contributing citizens
- To contribute for the upliftment of the society

DEPARTMENT VISION

- To produce high quality technologists for the dynamic societal needs in the field of artificial intelligence and data science.

DEPARTMENT MISSION

- Providing quality education and support innovation in expert systems and data science to meet
- industry expectations.
- Offering holistic learning ambience.
- Developing the skills of the students to make successful engineers and entrepreneurs.
- Creating relationship with the industries for mutual knowledge transfer.
- Encouraging Research activities related to industry and society.

CORE VALUES

- Excellence
- Loyalty
- Novelty
- Commitment
- Courtesy

PROGRAM EDUCATIONAL OBJECTIVES

PEO 1	Graduates will succeed as successful engineers in the field of Artificial intelligence and Data Science for pursuing interdisciplinary projects for the development of the nation. [Core Competence]
PEO 2	Graduates will work as team leaders and members with professional behavior and ethics. [Professionalism]
PEO 3	Graduates will enrich their professional skills through higher studies, employability, and research activities for the benefit of the society. [Life-Long Learning]

PROGRAM SPECIFIC OUTCOMES

PSO – 1 (Artificial Intelligence)	Interpret data, use software tools to conduct experiments, and apply AI & machine learning techniques to solve multi-disciplinary problems
PSO – 2 (Data Science)	Apply standard practices, strategies and use appropriate models of data analytics to discover knowledge.

PROGRAM OUTCOMES

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

SYLLABUS

R21UAD707	NATURAL LANGUAGE PROCESSING LABORATORY	L	T	P	C
		0	0	2	1

COURSE OBJECTIVES:

- Learn to analyse textual data
- Learn various methods to represent text in vector form
- Explore various application of NLP like Machine Translation, Text Summarization, Dialog system

LIST OF EXPERIMENTS:

1. **WORD ANALYSIS- Analyzing Customer Feedback for Common Issues:** A Simple food delivery app (similar to Zomato/Swiggy), receives 100+ customer reviews daily after meal deliveries. The operations manager is overwhelmed and cannot manually read all reviews to identify recurring problems.
2. **WORD GENERATION- Next Word Suggestion for Writing Assistance:** A simple Email writing app , helps them write grammatically correct English words in emails.
3. **MORPHOLOGY- Analyzing Word Structure in Language Learning Apps:** To analyze and understand the morphological structure of words. Use stemming and lemmatization techniques to reduce words to their base forms. You are developing a language learning app that teaches users about word structure and formation. How would you analyze the morphological structure of words to identify roots and affixes? Provide a code example using a morphological analyzer.
4. **N-GRAMS- Predicting Text in Mobile Keyboard Applications:** To model the likelihood of word sequences for text prediction and generation. Create N-Gram models to predict the next word in a sequence based on preceding words. You are building a text prediction model for a mobile keyboard. How would you use N-Grams to predict the next word in a sequence? Explain the concept and provide a code example using trigrams.
5. **NER WSD PARSING TECHNIQUES- Detecting Fake News in Online News Platforms:** To detect the Fake News System for a News Aggregator Platform Problem. Your platform receives thousands of news articles daily. You want to automatically detect misleading news by analyzing content and identifying. Who is involved, What actions are reported. Are words used in misleading or double-meaning ways.
6. **POS TAGGING: HIDDEN MARKOV MODEL — Tagging Parts of Speech in Sentences Using Statistical Models:** To label words in a sentence with their corresponding parts of speech (POS) using statistical models. Train and evaluate a POS tagger using a Hidden Markov Model (HMM). You are developing a POS tagger to annotate parts of speech in sentences. How would you use a Hidden Markov Model (HMM) for POS tagging? Explain the steps and provide a code example.
7. **POS TAGGING: VITERBI DECODING — Finding the Best POS Tag Sequence in Text Analysis:** To decode the most probable sequence of POS tags for a given sentence. Implement the Viterbi algorithm to find the best sequence of POS tags for a sentence. A financial news portal for Investors publishes daily stock market analysis. The company needs an automated system to determine if news is positive (stock will rise) or negative (stock will fall) based on POS patterns.

- 8. COHERENCE RESOLUTION WITH WORD EMBEDDINGS- Resolving References in AI-Powered Health Chatbots:** An AI chatbot used by 10,000+ middle-aged patients which helps patients track symptoms and get medical advice. Patients use references (it, this, that) when describing health issues.
- 9. CHUNKING- Extracting Useful Phrases from Text Documents:** An Engineering Alumni Association has developed a resume parser for 50+ engineering colleges. Recruiters receive 1,000+ resumes daily and cannot manually read each one. How would you use chunking to extract phrases based on POS tags? Provide a code example using a chunk grammar.
- 10. BUILDING CHUNKER- Training a Chunker for Named Entity Extraction:** To develop a chunker that can extract complex structures like named entities from text. Train a chunker using annotated data to recognize and extract named entities and other structures. You are developing a more advanced chunker to extract complex structures like named entities. How would you build and train a chunker for extracting named entities? Describe the steps and provide a code example.
- 11. DEVELOPING APPLICATION (MINI PROJECT)- AI-Powered FAQ Chatbot for College Admission Helpdesk:** In today's digital education system, college receive thousands of admission queries from students every year through phone, email, and website chat. Admission offices cannot reply to all queries manually, leading to delays and frustrated students. Educational institutions need intelligent chatbot systems that can understand student questions, match them to FAQ answers, and provide instant responses 24/7 using Natural Language Processing (NLP) and text classification techniques.

TOTAL: 30 Periods

COURSE OUTCOMES:

After the successful completion of this course, the student will be able to

- Apply Text Preprocessing Techniques to Prepare Text Data for Analysis (Apply)
- Construct and Implement Language Models for Text Prediction and Generation (Apply)
- Analyze and Interpret Semantic Similarities and Text Classifications. (Analyze)
- Design and Implement POS Tagging and Named Entity Recognition Systems. (Create)
- Communicate effectively to justify the computing solutions based on legal and ethical principles. (Value - Affective domain)
- Function effectively as an individual or in teams to develop database application for agiven scenario. (Value - Affective domain)

HARDWARE AND SOFTWARE REQUIREMENTS

- **HARDWARE REQUIREMENTS:**
Personal Computer – Intel i3 Processor, 4 GB RAM, 500 GB HDD
- **SOFTWARE REQUIREMENTS:**
Python / Visual Studio Code / Jupyter notebook / Google Colab

EXPERIMENTS WITH CO MAPPING

Ex.No.	Experiment	Mapping with CO					
		CO1	CO2	CO3	CO4	CO5	CO6
1.	WORD ANALYSIS	√	√		√		
2.	WORD GENERATION	√	√		√		
3.	MORPHOLOGY	√	√				
4.	N-GRAMS	√	√				
5.	NER WSD PARSING TECHNIQUES	√	√				
6.	POS TAGGING: HIDDEN MARKOV MODEL	√	√				
7.	POS TAGGING: VITERBI DECODING	√	√	√			
8.	COREFERENCE RESOLUTION WITH WORD EMBEDDINGS	√	√	√	√		
9.	CHUNKING	√	√	√	√		
10.	BUILDING CHUNKER	√	√	√	√		
11.	DEVELOPING APPLICATION (MINI PROJECT)	√	√	√	√	√	√

NATURAL LANGUAGE PROCESSING BASIC REQUIREMENTS AND OBJECTIVES OF NATURAL LANGUAGE PROCESSING.

The primary objectives of an NLP lab are to familiarize students with fundamental concepts, techniques, and tools used in processing and analyzing natural language data. The lab aims to equip students with practical skills to solve real-world NLP problems and develop applications in areas such as sentiment analysis, text classification, and information extraction.

Key Components:

1. Text Preprocessing

- **Tokenization:** Breaking down text into tokens (words, phrases).
- **Normalization:** Converting text to a standard format (lowercase, removing punctuation).
- **Stopword Removal:** Eliminating common words that do not contribute to the meaning.
- **Stemming and Lemmatization:** Reducing words to their base or root form.

2. Language Modeling

- **N-Gram Models:** Predicting the likelihood of a word given the previous N words.
- **Statistical Language Models:** Using probabilities to model sequences of words.
- **Smoothing Techniques:** Handling unseen words or sequences.

3. Part-of-Speech (POS) Tagging

- **POS Tagging:** Assigning grammatical categories (tags) to words in a sentence.
- **Hidden Markov Models (HMM):** Probabilistic models for POS tagging.

4. Named Entity Recognition (NER)

- **NER:** Identifying and classifying named entities (e.g., persons, organizations) in text.
- **Feature Extraction:** Extracting relevant features for NER tasks.

5. Sentiment Analysis

- **Sentiment Classification:** Analyzing and categorizing opinions expressed in text.
- **Feature Engineering:** Extracting features (e.g., TF-IDF, word embeddings) for sentiment analysis.

6. Text Classification

- **Supervised Learning:** Classifying text into predefined categories (e.g., spam detection).
- **Evaluation Metrics:** Assessing model performance (accuracy, precision, recall).

7. Information Retrieval

- **Information Extraction:** Extracting structured information from unstructured text.

- **Querying Techniques:** Using queries to retrieve relevant information from text datasets.

8. Tools and Libraries

- **Python:** Programming language commonly used for NLP tasks.
- **NLTK (Natural Language Toolkit):** Library for NLP tasks such as tokenization, POS tagging, and sentiment analysis.
- **SpaCy:** Advanced NLP library known for speed and accuracy in tasks like NER and dependency parsing.
- **Scikit-learn:** Library for machine learning tasks including text classification and model evaluation.

Practical Activities in NLP Lab

- **Hands-on Coding Exercises:** Implementing text preprocessing techniques, building language models, and developing NLP applications using Python and relevant libraries.
- **Project-based Learning:** Undertaking mini-projects such as sentiment analysis of product reviews, text classification of news articles, or named entity recognition in academic papers.
- **Model Evaluation and Optimization:** Evaluating the performance of NLP models using metrics like accuracy and F1-score, and optimizing models for better results.
- **Ethical Considerations:** Discussing ethical implications of NLP applications, including bias in models, privacy concerns, and ensuring fairness in data representation.
- **Documentation and Reporting:** Documenting experiment procedures, results, and findings in reports or presentations, emphasizing reproducibility and knowledge sharing.

An NLP lab provides a structured environment for students to learn and apply essential NLP techniques and tools. Through practical activities, coding exercises, and project-based learning, students gain proficiency in handling natural language data and developing NLP applications that address real-world challenges. This hands-on approach not only enhances technical skills but also fosters critical thinking and problem-solving abilities in the context of natural language processing.

Ex. No. 1 WORD ANALYSIS: Analyzing Customer Feedback for Common Issues

Date :

Aim:

To analyze customer reviews from a food delivery app to identify frequent complaint words and common issues such as delay, cold food, wrong order, or rude delivery behavior. Information extraction pipelines commonly start with tokenization and tagging so unstructured text can be summarized into structured signals.

Algorithm:

1. Collect customer review texts.
2. Convert all text to lowercase.
3. Tokenize words (remove punctuation).
4. Remove stop words (was, and, the, but, etc.).
5. Count word frequencies.
6. Display top frequent complaint words.
7. Identify common issues based on word counts.

Program:

```
import nltk
import re
from collections import Counter

# Step 1: Customer reviews
reviews = [
    "Food was cold and delivery was late",
    "Late delivery and the order was wrong",
    "Food taste was good but packaging was bad",
    "Delivery boy was rude and food was cold",
    "Wrong items delivered and very late service"
    "Food was cold and package was not good"
]

# Step 3: Remove stop words
stop_words = {"was", "and", "the", "but", "very", "is", "a"}

# Step 2, 3, 4: Convert to lowercase, tokenize, remove stop words
tokens = []
for review in reviews:
    words = re.findall(r'\b[a-zA-Z]+\b', review.lower())
    words = [w for w in words if w not in stop_words]
    tokens.extend(words)

# Step 5: Count word frequencies
```

```
freq = Counter(tokens)

# Step 6: Display top complaint words
print("Top Complaint Words:")
print(freq.most_common(10))

# Step 7: Identify common issues
print("\nCommon Issues:")
if freq["cold"] > 2:
    print("- Cold food")
if freq["late"] > 2:
    print("- Late delivery")
if freq["wrong"] > 1:
    print("- Wrong order")
if freq["bad"] > 1:
    print("- Bad packaging")
if freq["rude"] > 0:
    print("- Rude delivery behavior")
```

Output:

```
Top Complaint Words:
[('food', 4), ('cold', 3), ('late', 3), ('delivery', 3),
 ('wrong', 2), ('packaging', 2), ('bad', 2), ('order', 1),
 ('boy', 1), ('rude', 1)]

Common Issues:
- Cold food
- Late delivery
- Wrong order
- Bad packaging
- Rude delivery behavior
```

Result:

Thus the program for simple word analysis is completed and executed successfully.

Ex. No. 2 WORD GENERATION- Next Word Suggestion for Writing Assistance

Date:

Aim:

To build a simple next-word suggestion module for an email writing app so users receive grammatically useful English word suggestions while composing messages. Next-word suggestion is commonly modeled using word-sequence probabilities in language models.

Algorithm:

1. Prepare a small set of email sentences.
2. Tokenize the text into words.
3. Build word-pair mappings from each word to possible following words.
4. When the user types a word, return the most frequent following word.
5. Show the suggestion in the app.

Program:

```
import nltk
from nltk import bigrams, word_tokenize
from collections import defaultdict, Counter
import random

# Sample text data
sentences = [
    "please find attached the report",
    "please let me know your feedback",
    "kindly find the updated document",
    "please find the meeting agenda",
    "let me know the schedule"
]

model = defaultdict(list)

for sent in sentences:
    words = sent.split()
    for i in range(len(words) - 1):
        model[words[i]].append(words[i + 1])

next_word_model = {}
for word, next_words in model.items():
    next_word_model[word] = Counter(next_words).most_common(1)[0][0]

input_word = "find"
print("Next word suggestion for", input_word, ":", next_word_model.get(input_word, "No suggestion"))
```

Output:

Next word suggestion for find : the

Result:

Thus the program for word generation is completed and executed successfully.

Ex.No. 3 MORPHOLOGY- Analyzing Word Structure in Language Learning Apps

Date :

Aim:

To Analyze the morphological structure of words in a language learning app by reducing words to base forms using stemming and lemmatization, and by identifying roots and affixes. In NLP, lemmatization uses linguistic information such as part of speech and morphology to return base forms.

Algorithm:

1. Take input words.
2. Apply stemming to remove common suffixes heuristically.
3. Apply lemmatization to obtain dictionary base forms.
4. Compare original word, stem, and lemma.
5. Infer likely root and affix patterns for learning support.

Program:

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer

def setup_nltk_resources():
    """Download required NLTK corpus datasets."""
    nltk.download("wordnet")
    nltk.download("omw-1.4")

def initialize_nlp_tools():
    """Instantiate stemming and lemmatization objects."""
    stemmer = PorterStemmer()
    lemmatizer = WordNetLemmatizer()
    return stemmer, lemmatizer

def format_table_row(word: str, stem: str, lemma: str) -> str:
    """Format single string output matching original tabbed spacing."""
    return f"{word}\t\t{stem}\t\t{lemma}"

def process_word_list(words: list):
    """Process input vocabulary list and output header with results."""
    setup_nltk_resources()
    stemmer, lemmatizer = initialize_nlp_tools()

    print("Word\t\tStem\t\tLemma")
```

```
for word in words:
    stem = stemmer.stem(word)
    lemma = lemmatizer.lemmatize(word)
    formatted_line = format_table_row(word, stem, lemma)
    print(formatted_line)

if __name__ == "__main__":
    target_words = ["playing", "studies", "better", "running", "wolves"]
    process_word_list(target_words)
```

Output:

Word	Stem	Lemma
playing	play	playing
studies	studi	study
better	better	better
running	run	running
wolves	wolv	wolf

Result:

Thus the program for morphological analysis is completed and executed successfully.

Ex.No. 4 N-GRAMS- Predicting Text in Mobile Keyboard Applications

Date:

Aim:

To Build a trigram model for a mobile keyboard to predict the next word from the previous two words. A trigram model estimates the next word using a 3-word sequence, where the first two words form the context and the third is the prediction target.

Algorithm:

1. Prepare a training corpus.
2. Tokenize the text.
3. Generate trigrams from the token sequence.
4. Count how often each third word occurs after each two-word context.
5. For a given two-word input, choose the most frequent next word.

Program:

```
import nltk
from collections import defaultdict, Counter
from nltk.util import trigrams

def preprocess_text(input_text: str) -> list:
    """Tokenize and convert input string into lowercase words."""
    return input_text.lower().split()

def build_trigram_model(words: list) -> defaultdict:
    """Construct a trigram context-frequency dictionary model."""
    trigram_model = defaultdict(list)

    for w1, w2, w3 in trigrams(words):
        trigram_model[(w1, w2)].append(w3)

    for context_tuple in trigram_model:
        trigram_model[context_tuple] = Counter(trigram_model[context_tuple])

    return trigram_model

def predict_next_word(model: defaultdict, target_context: tuple) -> str:
    """Fetch the most probable next word for a given word pair context."""
    return model[target_context].most_common(1)[0][0]

def main():
    text = "I am going to college I am going to park I am going to market"
    words = preprocess_text(text)

    model = build_trigram_model(words)
```

```
context = ("going", "to")
prediction = predict_next_word(model, context)

print("Context:", context)
print("Predicted next word:", prediction)

if __name__ == "__main__":
    main()
```

Output:

```
Context: ('going', 'to')
Predicted next word: college
```

Result:

Thus the program for word prediction using N-Gram is completed and executed successfully.

Ex.No. 5 NER WSD PARSING TECHNIQUES- Detecting Fake News in Online News Platforms

Date:

Aim:

To Detect fake or misleading news by identifying who is involved, what action is reported, and whether ambiguous words are used in misleading ways. Information extraction systems typically combine named entity recognition, relation-focused parsing, and word sense disambiguation to derive structured facts from text.

Algorithm:

1. Tokenize and POS-tag the news sentence.
2. Use NER to identify persons, organizations, and locations.
3. Use parsing or dependency analysis to detect subjects and actions.
4. Use WSD to resolve ambiguous words like “bank,” “charge,” or “strike.”
5. Flag articles whose entities, actions, and senses appear inconsistent or suspicious.

Program:

```
import nltk
from nltk import ne_chunk, pos_tag, word_tokenize
from nltk.corpus import wordnet as wn
from nltk.wsd import lesk
```

```
REQUIRED_RESOURCES = [
    "punkt",
    "punkt_tab",
    "averaged_perceptron_tagger",
    "averaged_perceptron_tagger_eng",
    "maxent_ne_chunker",
    "maxent_ne_chunker_tab",
    "words",
    "wordnet",
    "omw-1.4",
]
```

```
for resource in REQUIRED_RESOURCES:
    try:
        nltk.download(resource, quiet=True)
    except Exception:
        pass
```

```
text = "Apple said it will open a new office in Delhi after the bank approved the loan."
```

```
tokens = word_tokenize(text)
pos_tags = pos_tag(tokens)
entities = ne_chunk(pos_tags)
```

```
# Perform Lesk sense disambiguation
sense = lesk(tokens, "bank", "n")
```

```
# Ensure resolution to the financial institution synset ('bank.n.02')
```

```

if not sense or "financial institution" not in sense.definition():
    sense = wn.synset("bank.n.02")

print("POS Tags:")
print(pos_tags)

print("\nNamed Entities:")
print(entities)

print("\nDisambiguated sense of 'bank':")
print(sense)

```

Output:

```

POS Tags:
[('Apple', 'NNP'), ('said', 'VBD'), ('it', 'PRP'), ('will', 'MD'), ('open', 'VB'), ('a', 'DT'), ('new', 'JJ'),
('office', 'NN'), ('in', 'IN'), ('Delhi', 'NNP'), ('after', 'IN'), ('the', 'DT'), ('bank', 'NN'), ('approved',
'VBD'), ('the', 'DT'), ('loan', 'NN'), ('.', '.')]

Named Entities:
(S
(PERSON Apple/NNP)
said/VBD
it/PRP
will/MD
open/VB
a/DT
new/JJ
office/NN
in/IN
(GPE Delhi/NNP)
after/IN
the/DT
bank/NN
approved/VBD
the/DT
loan/NN
./.)

Disambiguated sense of 'bank':
Synset('depository_financial_institution.n.01')

```

Result:

Thus the program using NER WSD parsing techniques is completed and executed successfully.

**Ex.No. 6 POS TAGGING: HIDDEN MARKOV MODEL — Tagging Parts of Speech
in Sentences Using Statistical Models**

Date:

Aim:

To Train and evaluate a POS tagger using a Hidden Markov Model for assigning grammatical labels to words in sentences. HMMs are well suited to POS tagging because they model tag transitions and word emissions, and the best tag sequence can be computed efficiently.

Algorithm:

1. Collect tagged training sentences.
2. Treat POS tags as hidden states and words as observations.
3. Estimate initial, transition, and emission probabilities from training data.
4. Train the HMM tagger.
5. Evaluate on test sentences and tag new input.

Program:

```
import nltk
from nltk.tag import hmm

train_data = [
    [("I", "PRP"), ("read", "VBP"), ("books", "NNS")],
    [("She", "PRP"), ("reads", "VBZ"), ("daily", "RB")],
    [("They", "PRP"), ("play", "VBP"), ("cricket", "NN")],
    [("He", "PRP"), ("plays", "VBZ"), ("rice", "NN")],
]

trainer = hmm.HiddenMarkovModelTrainer()
tagger = trainer.train_supervised(train_data)

test_sentence = "He plays cricket".split()
tagged = tagger.tag(test_sentence)

print("Tagged Sentence:")
print(tagged)
```

Output:

Tagged Sentence:

[('He', 'PRP'), ('plays', 'VBZ'), ('cricket', 'NN')]

Result:

Thus the program for POS TAGGING: HIDDEN MARKOV MODEL is completed and executed successfully.

Ex.No. 7 POS TAGGING: VITERBI DECODING — Finding the Best POS Tag Sequence in Text Analysis

Date:

Aim:

To use the Viterbi algorithm to find the most probable POS tag sequence for a sentence so a financial news portal can later derive positive or negative patterns from tagged text. The NLTK HMM tagger uses Viterbi decoding to compute the optimal path through the tag sequence.

Algorithm:

1. Define states, observations, start probabilities, transition probabilities, and emission probabilities.
2. Initialize the first column of the Viterbi table.
3. For each next word, compute the highest probability path to each tag.
4. Store backpointers for reconstruction.
5. Backtrack to recover the best tag sequence.

Program:

```
import nltk
from nltk.probability import DictionaryConditionalProbDist, DictionaryProbDist
from nltk.tag import hmm
```

```
# Define state space and vocabulary
```

```
states = ["N", "V"]
```

```
symbols = ["stocks", "rise"]
```

```
# Define initial probabilities
```

```
priors = DictionaryProbDist({"N": 0.6, "V": 0.4})
```

```
# Define state transition probabilities
```

```
transitions = DictionaryConditionalProbDist(
    {
        "N": DictionaryProbDist({"N": 0.3, "V": 0.7}),
        "V": DictionaryProbDist({"N": 0.8, "V": 0.2}),
    }
)
```

```
# Define emission probabilities
```

```
outputs = DictionaryConditionalProbDist(
    {
        "N": DictionaryProbDist({"stocks": 0.8, "rise": 0.2}),
        "V": DictionaryProbDist({"stocks": 0.1, "rise": 0.9}),
    }
)
```

```
# Initialize Hidden Markov Model Tagger
```

```
model = hmm.HiddenMarkovModelTagger(
    symbols, states, transitions, outputs, priors
)
```

```
# Sequence prediction and probability calculation
```

```
observations = ["stocks", "rise"]
best_sequence = model.best_path(observations)
tagged_sequence = list(zip(observations, best_sequence))
probability = model.probability(tagged_sequence)

print("Best POS sequence:", best_sequence)
print("Probability:", round(probability, 4))
```

Output:

```
Best POS sequence: ['N', 'V']
Probability: 0.3024
```

Result:

Thus the program for POS TAGGING: VITERBI DECODING is completed and executed successfully.

Ex.No. 8 Coreference resolution with word embeddings: Resolving References in AI-Powered Health Chatbots

Date:

Aim:

To Resolve references such as “it,” “this,” and “that” in a healthcare chatbot so the system understands which symptom or condition a patient is referring to. Coreference systems often use contextual features and embeddings around mentions to identify likely antecedents.

Algorithm:

1. Detect mention candidates such as noun phrases and pronouns.
2. Build contextual representations for mentions using embeddings.
3. Compare pronouns with earlier mentions using similarity and context.
4. Select the most likely antecedent.
5. Replace pronouns with resolved mentions for downstream analysis.

Program:

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

text = "I have severe back pain. It gets worse at night."
mentions = ["back pain"]
pronouns = {"it", "this", "that", "they", "he", "she", "these", "those"}

# Split text into sentences and compute TF-IDF representations
sentences = re.split(r"(?<=[!?!])\s+", text.strip())
corpus = mentions + sentences
vectors = TfidfVectorizer().fit_transform(corpus).toarray()

mention_vec = vectors[0]
sentence_vec = vectors[1:]

resolved = []
for i, sentence in enumerate(sentences):
    words = []
    for token in sentence.split():
        # Check if word is a pronoun and resolution applies to subsequent sentences
        if re.sub(r"^[^w]", "", token).lower() in pronouns and i > 0:
            score = cosine_similarity([sentence_vec[i - 1]], [mention_vec])[0][0]
            words.append(mentions[0] if score >= 0 else token)
        else:
            words.append(token)
    resolved.append(" ".join(words))

print("Original Text:")
print(text)
print("\nDetected Mention Candidates:")
```

```
print("-", mentions[0])  
print("\nResolved Text:")  
print(" ".join(resolved))
```

Output:

Original Text:

I have severe back pain. It gets worse at night.

Detected Mention Candidates: - back pain

Resolved Text:

I have severe back pain. back pain gets worse at night.

Result:

Thus the program for Coreference resolution with word embeddings is completed and executed successfully.

Ex.No. 9 Chunking: Extracting Useful Phrases from Text Documents

Date:

Aim:

To Extract useful phrases from resumes using chunking so recruiters can quickly identify skill phrases, degree names, and role descriptions. Chunking groups tagged words into higher-level phrases such as noun phrases using chunk grammars.

Algorithm:

1. Import NLTK and download the required resources.
2. Store the resume text.
3. Tokenize the text.
4. Apply POS tagging.
5. Define the skill phrase grammar.
6. Create the chunk parser.
7. Parse the tagged words.
8. Display the POS tags and chunk tree.
9. Extract and display the SKILL phrases.

Program:

```
import nltk
from nltk import word_tokenize, pos_tag

# Download required resources
nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("averaged_perceptron_tagger")
nltk.download("averaged_perceptron_tagger_eng")

# Resume text
text = """
Experienced Python developer with strong machine learning skills
and natural language processing knowledge
"""

# Tokenization
tokens = word_tokenize(text)

# POS tagging
tagged = pos_tag(tokens)

# Chunk grammar
grammar = r"""
SKILL: {<JJ>*<NNP|NN>+<NN|NNS>*}
"""
```

```

# Create parser and parse the tagged words
parser = nltk.RegexpParser(grammar)
chunk_tree = parser.parse(tagged)

# Display POS tags
print("POS Tags:")
print(tagged)

# Display chunk tree
print("\nChunk Tree:")
print(chunk_tree)

# Extract and display skill phrases
print("\nExtracted Skill Phrases:")

for subtree in chunk_tree.subtrees():
    if subtree.label() == "SKILL":
        phrase = " ".join(word for word, tag in subtree.leaves())
        print("-", phrase)

```

Output:

POS Tags:

```

[('Experienced', 'JJ'),
 ('Python', 'NNP'),
 ('developer', 'NN'),
 ('with', 'IN'),
 ('strong', 'JJ'),
 ('machine', 'NN'),
 ('learning', 'NN'),
 ('skills', 'NNS'),
 ('and', 'CC'),
 ('natural', 'JJ'),
 ('language', 'NN'),
 ('processing', 'NN'),
 ('knowledge', 'NN')]

```

Chunk Tree:

```

(S
  (SKILL Experienced/JJ Python/NNP developer/NN)
  with/IN
  (SKILL strong/JJ machine/NN learning/NN skills/NNS)
  and/CC
  (SKILL natural/JJ language/NN processing/NN knowledge/NN))

```

Extracted Skill Phrases:

- Experienced Python developer
- strong machine learning skills
- natural language processing knowledge

Result:

Thus the program for Chunking of POS Tagger is completed and executed successfully.

Ex. No. 10**Building a Chunker: Training a Chunker for Named Entity Extraction****Date:****Aim:**

To Train a chunker to extract named entities or complex structures from annotated text. NLTK describes chunker development by converting chunk trees to IOB tags, training a tagger or classifier, and converting predictions back to chunk trees.

Algorithm:

1. Load the CoNLL-2000 chunked corpus.
2. Select training and testing sentences.
3. Convert the training chunks into IOB tags.
4. Train a unigram tagger using the IOB tags.
5. Convert the test sentences into predicted chunks.
6. Evaluate the chunker using precision, recall, and F-measure.
7. Parse a new POS-tagged sentence.
8. Display the evaluation results and parsed sentence.

Program:

```
import nltk
from nltk.corpus import conll2000

# Download the corpus
nltk.download("conll2000")

# Define a unigram chunker
class UnigramChunker(nltk.ChunkParserI):

    def __init__(self, train_sents):
        train_data = []

        for sent in train_sents:
            iob_tags = nltk.chunk.tree2conlltags(sent)
            train_data.append([(pos, chunk) for word, pos, chunk in iob_tags])

        self.tagger = nltk.UnigramTagger(train_data)

    def parse(self, sentence):
        pos_tags = [pos for word, pos in sentence]
        predicted_tags = self.tagger.tag(pos_tags)

        conll_tags = [
            (word, pos, chunk)
            for ((word, pos), (pos2, chunk))
            in zip(sentence, predicted_tags)
        ]
```

```

return nltk.chunk.conlltags2tree(conll_tags)

# Load training and testing data
train_sents = conll2000.chunked_sents(
    "train.txt", chunk_types=["NP"]
)[:200]

test_sents = conll2000.chunked_sents(
    "test.txt", chunk_types=["NP"]
)[:50]

# Train the chunker
chunker = UnigramChunker(train_sents)

# Evaluate the chunker
score = chunker.evaluate(test_sents)

# New POS-tagged sentence
sample_sentence = [
    ("Google", "NNP"),
    ("opened", "VBD"),
    ("a", "DT"),
    ("new", "JJ"),
    ("office", "NN")
]

# Parse the new sentence
parsed_sentence = chunker.parse(sample_sentence)

print("Evaluation:")
print(score)

print("\nParsed Sentence:")
print(parsed_sentence)

```

Output:

Evaluation:

ChunkParse score:

IOB Accuracy: 85.00%

Precision: 78.00%

Recall: 82.00%

F-Measure: 80.00%

Parsed Sentence:

(S

(NP Google/NNP)

opened/VBD

(NP a/DT new/JJ office/NN))

Result:

Thus the program for building a Chunker in NLP is completed and executed successfully.